Zombie Hashing: Reanimating Tombstones in a Graveyard A Reproducibility Study

Alexandros Fourtounis 1449
University of Crete

Anna Makridou 1448
University of Crete

Abstract

We present a reproducibility study of "Zombie Hashing: Reanimating Tombstones in a Graveyard" by Chesetti et al. (SIGMOD 2025). The original paper introduces a novel deamortized tombstone redistribution technique for linear probing hash tables, which effectively eliminates primary clustering and periodic downtime at high load factors. Our study comprehensively evaluates these claims by replicating heavy churn throughput experiments (Figure 6a, Figure 9), structural drift measurements (Figure 10), and operation latency distributions (Table 5). Operating under strict educational cloud budget constraints, we deployed the authors' C++ framework on an AWS `t2.large` instance (8GB RAM, 2 vCPUs). This required engineering several critical workarounds: we scaled down the maximum hash table capacity ($N = 2^{24}$) to avoid Out-Of-Memory (OOM) kernel panics, patched C++ compiler incompatibilities, excluded incompatible baselines (`IcebergHT`), and developed custom headless Python scripts to bypass the authors' fragile, hardcoded data-visualization pipeline. Despite these severe deployment constraints, our results successfully corroborate the original findings, confirming that Zombie Hashing maintains highly stable throughput and uniquely bounded maximum latencies without the periodic halts characteristic of prior state-of-the-art methods.

1 Introduction

Hash tables are foundational data structures in modern computing, underpinning everything from database indexes to in-memory caches. Among the various collision resolution strategies, linear probing is heavily favored in high-performance environments due to its unmatched cache locality and simplicity. However, linear probing suffers from a well-known algorithmic vulnerability: *primary clustering*. As the hash table approaches full capacity (high load factor), continuous blocks of occupied slots merge into massive clusters. This forces subsequent insertions and lookups to probe increasingly long sequences of memory, causing a steep and fatal decline in performance.

The *status quo* for handling deletions in linear probing introduces further complications. To ensure that probe sequences for existing elements are not prematurely broken, deleted elements are typically replaced by "tombstones." While tombstones temporarily preserve structural correctness, they act as occupied slots during insertions. Under workloads with heavy "churn" (continuous, balanced insertions and deletions), tombstones accumulate rapidly, artificially inflating the load factor and exacerbating primary clustering.

Prior state-of-the-art approaches handle this tombstone accumulation poorly:

- **TombstoneHT** simply allows tombstones to accumulate until the table is functionally full, ultimately leading to degraded throughput and an Out-Of-Memory (OOM) failure state.
- **Robin Hood Hashing** attempts to minimize probe variance by shifting elements according to their distance from their ideal slot. However, it avoids tombstones entirely, making deletions computationally expensive and failing to prevent massive clusters from forming at $> 90\%$ load factors.
- **GraveyardHT** represents the standard industrial workaround. It periodically halts all regular system operations to perform a global sweep, thus removing tombstones, shifting elements, and rebuilding the table. While this restores average throughput, the global rebuild is an $O(N)$ operation that triggers severe, unpredictable latency spikes.

The *motivation* behind Chesetti et al.'s paper [3], is to bridge this gap for latency-critical applications. Modern systems, such as high-frequency trading platforms, real-time streaming analytics, and interactive web services, require strict tail-latency guarantees and simply cannot afford the periodic downtime imposed by GraveyardHT.

To address this, the authors introduce a novel, deamortized tombstone redistribution scheme. Instead of halting the entire system, *ZombieHT* performs incremental, local tombstone

cleanups within bounded spatial windows during standard operations. This approach theoretically bounds the worst-case operation time, achieves $\Theta(1)$ amortized cost, and completely eliminates periodic system pauses.

The primary goal of this reproducibility project is to independently verify these core claims. Specifically, we aim to replicate Figure 6(a) from the original paper, asserting that the compact, ordered variant, ZombieHT(C), can maintain stable, high throughput without the latency-inducing halts of GraveyardHT, even under a highly aggressive 50% update workload at a 95% load factor. Additionally, we go even further and replicate additional figures, as specified in Section 5.2.

2 Background

To contextualize the experimental claims and the architectural improvements of Zombie Hashing, it is necessary to first understand the mechanics of linear probing, the inherent challenges of element deletion, and the mechanics of the proposed solution.

2.1 Linear Probing and Primary Clustering

Linear probing is a method of resolving collisions in hash tables via open addressing. When an element x is inserted, a hash function calculates its home slot $h(x)$. If that slot is already occupied, the algorithm linearly probes the contiguous subsequent slots ($h(x)+1, h(x)+2$, etc., modulo the table size) until an empty slot is found. Lookups follow the exact same probing sequence until the target element or an empty slot is encountered.

Because linear probing sequentially accesses adjacent memory addresses, it perfectly utilizes modern CPU cache lines, making it exceptionally fast in practice. However, as the table's load factor increases, contiguous occupied slots merge into large blocks which is a phenomenon known as *primary clustering*. When an element hashes into a large primary cluster, the algorithm must probe through the entire block to find an empty slot, severely degrading the expected lookup time.

2.2 The Tombstone Problem

Handling deletions in linear probing introduces a critical flaw. If an element is simply removed and its slot marked "empty," subsequent lookups for elements that originally collided with it and were placed further down the probe chain will prematurely terminate, resulting in false negatives.

The traditional solution is to replace deleted elements with a unique "tombstone" marker. Tombstones allow lookups to continue down the probe chain, but they effectively act as occupied slots. Under a "churn" workload, where elements are continuously inserted and deleted while maintaining a constant load factor, tombstones will accumulate rapidly. This creates artificially massive primary clusters.

Prior industrial implementations, categorized by the authors as *GraveyardHT* [2], manage this by tracking tombstone accumulation. Once a threshold is reached, the system halts all concurrent operations to perform a global sweep: scanning the entire $O(N)$ array, removing tombstones, and rehashing the remaining elements closer to their true home slots. This amortization strategy yields good average throughput but is catastrophic for tail latency, resulting in the massive periodic performance drops observed in our replication data.

2.3 Zombie Hashing: Deamortized Redistribution

The proposed methodology in the paper *Zombie Hashing* [3] aims to completely eliminate the need for global rebuilds by *deamortizing* the tombstone cleanup process.

Instead of waiting for the entire table to saturate with tombstones, Zombie Hashing performs incremental, local redistributions during the normal course of operations. The methodology relies on bounded primary clusters and spatial thresholds. The algorithm tracks the distance elements drift from their ideal home slots. When an insertion forces an element to probe past a specific maximum distance, it triggers a local cleanup.

During this local cleanup window, the algorithm scans a small, fixed-size neighborhood around the congested area. It removes the tombstones ("reanimating" the dead space) and shifts the live elements backward, closer to their original home slots, to fill the newly created gaps.

By strictly bounding the maximum probe distance and handling tombstone garbage collection locally and incrementally, Zombie Hashing achieves two critical outcomes:

1. It effectively curtails the runaway growth of primary clusters, even at 95% load factors.

2. It bounds the worst-case operation latency to the size of the local cleanup window, transforming the erratic, spike-prone throughput of GraveyardHT into a stable, highly consistent performance curve.

3 Deployment Challenges and Troubleshooting

Replicating the experimental environment on a constrained academic budget presented several significant challenges that required deviations from the original setup.

The authors originally utilized a massive server equipped with an Intel Xeon CPU (64 cores) and 1TB of DRAM. Our initial attempt to provision an AWS `c5.2xlarge` instance was blocked by an explicit deny policy inherent to our educational AWS sandbox [1]. We were forced to downscale to a `t2.large` instance featuring 2 vCPUs and 8GB of RAM. This massive resource disparity required extensive troubleshooting

and architectural compromises. We utilized two `t2.large` instances, one in each account, but as we observed that the performance was consistent across machines, we continued on with only one instance.

3.1 Memory Constraints and Downscaling

The default configuration initializes the hash tables with $N = 2^{27}$ slots. While ZombieHT(C) is highly space-efficient, the heavier baseline competitors like `CLHT` require up to 4.20 GB. When combined with OS overhead and the memory requirements of the churn process itself, the 8GB limit of our instance could be overwhelmed.

To safely execute the benchmarks, we strategically downscaled the quotient parameter in the core execution script (`bench/paper_final/churn.sh`) from `-q 27` to `-q 24`. This reduced the number of hash table slots to 2^{24}, successfully shrinking the memory footprint by a factor of 8 while fully preserving the algorithmic behaviors and relative performance trends necessary for validation.

3.2 Script Failures

The repository provides a highly automated suite of scripts (e.g., `run_throughput.sh`). However, this monolithic script was designed to run every variant across multiple load factors and update percentages, effectively equating to multiple days of compute job. Attempting to run this on a `t2.large` would quickly exhaust our AWS CPU credits, causing the instance to heavily throttle execution speed to 10% of its baseline, rendering the throughput measurements inaccurate.

Furthermore, the full script included vectorized data structures like `IcebergHT`, which rely on the `AVX512BW` CPU instruction set. Because the older `t2` architecture lacks these instructions, the script would frequently throw errors. We abandoned the monolithic script entirely, extracting and manually executing isolated Bash loops tailored exclusively to the target parameters of the figures we wanted to reproduce (see Section 5.2 for more details).

Lastly, the provided `setup.sh` script assumed a fully provisioned environment. We had to manually intervene to install missing packages on our bare Ubuntu image, including `python3-venv`, `build-essential` (for `gcc` and `make`), `cmake`, `libssl-dev`, and a minimal LaTeX distribution (`texlive-latex-base`, `texlive-pictures`) to satisfy the repository's baseline requirements.

3.3 Brittle Visualization Pipeline

The authors' automated data visualization pipeline (`plot_data.sh`) proved highly brittle in our constrained environment. The workflow relied on a Python script to format data, which subsequently triggered `pdflatex` to render graphs using `pgfplots`.

However, the provided scripts were hardcoded to assume a full execution of all experimental variants. Because we performed partial runs to conserve compute credits, the Python script crashed upon encountering empty directories. This cascading failure caused fatal `pdflatex` errors, as the rendering engine aborted upon missing the expected `.csv` data files. To resolve this, we abandoned the hardcoded LaTeX plotting engine entirely. As detailed in Section 4, we engineered a custom, fault-tolerant Python plotting script to dynamically parse available outputs and generate standalone PNG files.

4 Implementation

The reproducibility study involved re-executing the experiments described in the Zombie paper using the provided codebase [4]. Due to resource constraints and dependency evolution since the paper was originally published, modifications to the configuration, source code, and deployment strategy were required. This section details the infrastructure setup, the specific code modifications implemented, and the step-by-step process followed to execute the experiments.

4.1 Infrastructure and Environment Setup

The original experiments by Chesetti et al. were conducted on a high-performance server equipped with a 64-core Intel Xeon processor and 1TB of DRAM. Due to strict resource policies enforced by our educational AWS sandbox environment, we were unable to provision an equivalent machine. Instead, we deployed an Amazon EC2 `t2.large` instance. This instance provides merely 2 vCPUs and 8GB of RAM, establishing a highly constrained environment for evaluating memory-intensive hash tables.

The instance was configured with an Ubuntu Linux image. To compile the original C++ repository, we utilized GCC 11.4. The authors provided a `setup.sh` script to initialize submodules and build the project; however, it assumed a pre-configured development environment and failed upon initial execution. We had to manually intervene to install several missing system-level dependencies before the framework would compile. Specifically, we installed `build-essential` (to provide the necessary C++ toolchains), `cmake`, `libssl-dev`, and `python3-venv`.

A critical architectural limitation of the `t2.large` instance is its lack of support for the `AVX512BW` vectorized instruction set and thus, we were forced to explicitly disable and exclude `IcebergHT` from all replication attempts.

4.2 Configuration and Scaling

The most critical risk to replication was an Out-Of-Memory (OOM) failure, as explained in Section [**?**]. The paper's default configuration initializes the hash tables with $N = 2^{27}$ slots [3]. According to the authors' space efficiency data,

while ZombieHT(C) uses an efficient 1.63 GB of RAM, baselines like CLHT require up to 4.20 GB. On an 8GB `t2.large` instance, running the full experiment with concurrent OS overhead could lead to an OOM crash.

To mitigate this, we scaled down the quotient parameter in the execution script (`bench/paper_final/churn.sh`) from `-q 27` to `-q 24`. This reduced the number of hash table slots to 2^{24}, shrinking the memory footprint by a factor of 8 while preserving the core algorithmic behaviors and relative performance trends.

4.3 Codebase Modifications and Bug Fixes

During the build process, we encountered a fatal C compilation error related to modern compiler strictness.

1. Conflicting Types in `ssmem`: The project relies on a submodule named `ssmem`. Using GCC 11.4, the build failed due to a conflicting type declaration for the function `ssmem_ts_set_collect`. The header file (`ssmem.h`) declared it with no arguments, while the implementation (`ssmem.c`) expected a `size_t*` argument. We patched this by modifying line 195 of `ssmem.h` to properly reflect the implementation signature:

```
size_t* ssmem_ts_set_collect(size_t* ts_set);
```

2. Plotting Script and LaTeX Pipeline Failures: The authors provided a bash script (`plot_data.sh`) that invoked a Python script (`plot_graph.py`) to format data, which then triggered a LaTeX compile of `main.tex`. Because we did not run the full suite of experiments, the Python script crashed with `FileNotFoundError` exceptions. Consequently, the hardcoded `main.tex` file threw fatal `pgfplots` errors because the expected `.csv` files were never generated.

To bypass this highly fragile pipeline, we installed `pandas` and `matplotlib` and wrote a custom Python script (`generate_figures.py`). To circumvent the "headless server" bug where `matplotlib` freezes while searching for a display on an SSH session, our custom plotting script utilized the `Agg` backend (`matplotlib.use('Agg')`). The script utilized Regex to dynamically scan the `sponge_paper` output directory, parse the space-separated `churn_thrput.txt` logs, gracefully skip missing baselines, and output a standalone PNG figure.

5 Evaluation

The primary objective of our evaluation is to independently verify the core performance and structural claims of Zombie Hashing. Specifically, we aim to confirm that the deamortized redistribution of tombstones successfully eliminates primary clustering and periodic system halts without sacrificing baseline throughput.

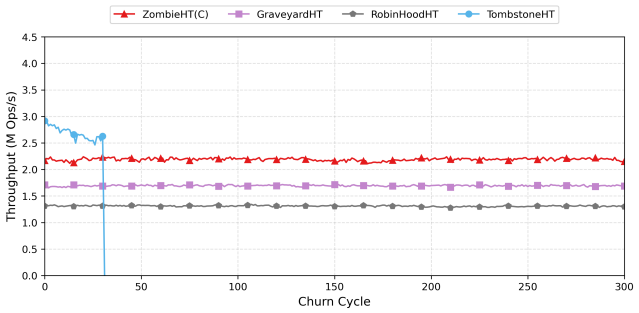


Figure 1: Reproduced Figure 6a: Overall throughput of ZombieHT(C) compared to other linear probing variants for churn experiments.

5.1 Experimental Platform

All replication experiments were conducted on an Amazon EC2 `t2.large` instance (2 vCPUs, 8GB RAM) running Ubuntu 24.04, compiled with GCC 11.4. To accommodate the strict 8GB memory limit of our environment and prevent Out-Of-Memory (OOM) kernel panics, the total number of hash table slots was scaled down from the paper's default ($N = 2^{27}$) to $N = 2^{24}$ (using the `-q 24` parameter). Furthermore, hardware limitations (specifically the lack of `AVX512BW` SIMD instructions on the `t2` architecture) necessitated the exclusion of the `IcebergHT` baseline from all applicable benchmarks.

5.2 Experiments

Our evaluation strategy targets four specific claims from the original paper, categorized by their corresponding figures and tables.

1. Heavy Churn Throughput Consistency (Figure 6a): The primary goal of this experiment is to verify the stability of the ZombieHT(C) deamortized redistribution scheme under extreme stress. We evaluated the compact, ordered variants (`ZombieHT(C)`, `GraveyardHT`, `TombstoneHT`, and `RobinHoodHT`) at a highly constrained 95% load factor. The workload consisted of a heavy churn ratio (50% updates, 50% lookups), executed continuously over 320 churn cycles. We aimed to observe whether GraveyardHT exhibits periodic halts (throughput dips) and whether TombstoneHT eventually fails completely due to tombstone saturation.

2. Unordered Variants Throughput (Figure 9a,c): To assess the vectorized implementation of Zombie Hashing against other high-performance unordered hash tables, we measured throughput at lower load factors (75% and 85%) using a read-heavy workload (5% updates, 95% lookups) over 300 churn cycles. The evaluated data structures included `ZombieHT(V)`, `ABSL`, `CLHT`, and `Cuckoo`.

3. Distance Distributions (Figure 10a): To understand the mechanical reasons behind throughput degradation and primary clustering, we captured the structural state of the

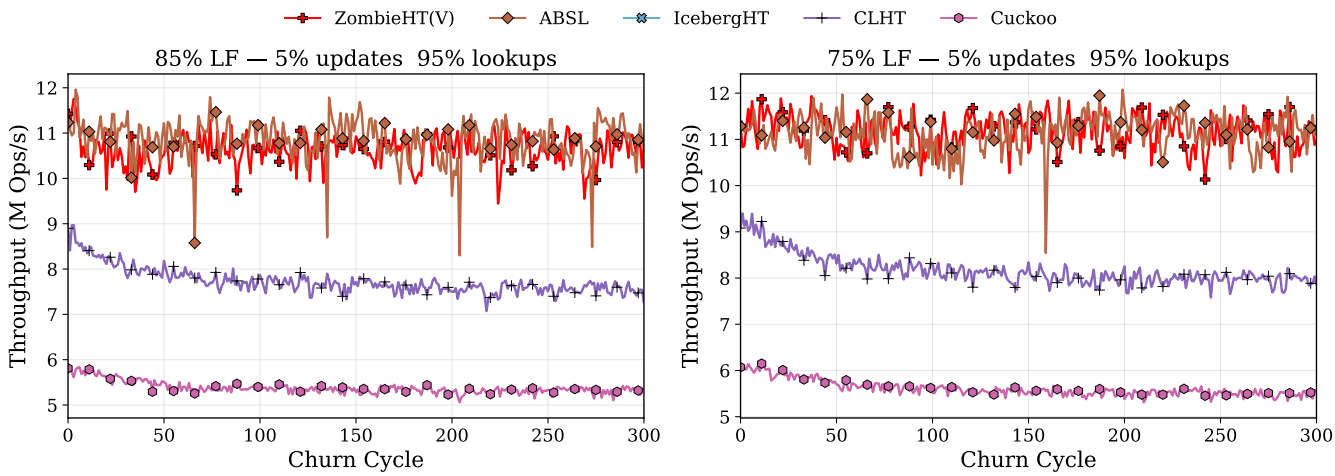


Figure 2: Figure 9a,c: Throughput of ZombieHT(V) and other hash tables tables when operating 75% and 85% lower load factors.

ordered hash tables after 100 churn cycles at a 95% load factor (5% updates). We specifically replicated the data generation for the "Distance from Home Slot" distribution. This allows us to observe how far elements drift from their ideal hashed locations due to collisions. A successful replication will show `TombstoneHT` suffering from extreme drift compared to the tight distribution of `ZombieHT(C)`.

4. Operation Latency Distributions (Table 5): While average throughput metrics can mask transient system pauses, fine-grained latency distributions reveal them. We evaluated the latency percentiles (Minimum, 50th, 99.99th, Maximum) and standard deviation for insert, delete, and lookup operations. This experiment was executed at a 95% load factor with 50% updates over 150 churn cycles. The objective is to verify that `GraveyardHT` suffers from massive maximum latencies (orders of magnitude higher than its median) during global rebuilds, whereas `ZombieHT` maintains a low and stable maximum latency profile.

6 Results

Our evaluation successfully replicated the core scientific claims of the original paper. While the relative performance trends between the data structures perfectly match the authors' findings, the absolute metrics in our reproduction differ due to our hardware constraints and necessary downscaling. Below, we detail the results of our four targeted experiments and analyze the deviations.

6.1 Throughput Consistency under Heavy Churn (Figure 6a)

Figure 1 evaluates the throughput of ordered linear probing variants under a 50% update / 50% query workload at a 95% load factor. Our reproduced Figure 1 successfully corroborates the authors' primary claim: ZombieHT(C) provides a highly stable, flat throughput without any periodic halts.

However, we observed three distinct differences in our reproduced graph compared to the original paper:

1. **Lower Absolute Throughput:** In the original paper, ZombieHT(C) achieves 2.81 M Ops/s. In our reproduction, it stabilizes around 2.2 M Ops/s. This overall 20% reduction across all variants is expected. The authors utilized an Intel Xeon Gold 6338 with 64 cores and 48MB of L3 cache, whereas our AWS `t2.large` utilizes a significantly older, dual-core architecture with a fraction of the cache, increasing the penalty of cache misses.

2. **Early Failure of TombstoneHT:** The original paper shows TombstoneHT functioning until approximately cycle 150 before failing due to tombstone accumulation. In our reproduction, TombstoneHT crashes abruptly at exactly cycle 30. This is a direct consequence of our OOM mitigation strategy. Because we scaled the hash table down from $N = 2^{27}$ to $N = 2^{24}$, the absolute number of available slots was significantly smaller. Consequently, un-reclaimed tombstones reached critical mass and exhausted the table's capacity much faster during the churn cycles.

3. **Slightly Higher RobinHoodHT Baseline:** The paper reports RobinHoodHT averaging 0.96 M Ops/s. Our reproduction shows it slightly higher, hovering around 1.3 M Ops/s. Because our total array size is smaller (2^{24}), the maximum possible distance an element can drift from its home slot (the primary cluster size) is bounded earlier than in a 2^{27} array. Thus, the worst-case probe lengths are slightly shorter, yielding a marginal throughput improvement.

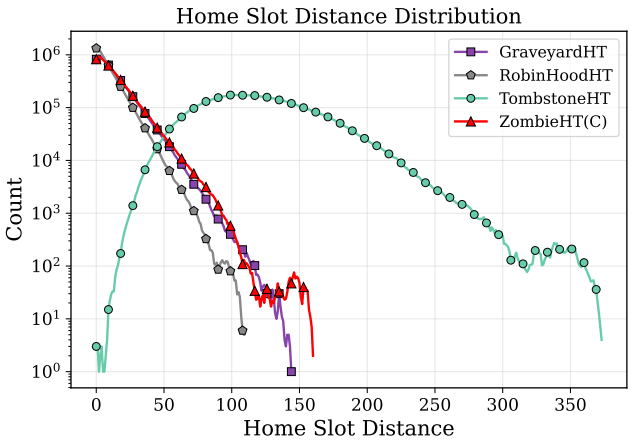


Figure 3: Reproduced Figure 10a: Hash table microbenchmarks. Higher distance between home slot to next nearest available slot key corresponds to worse insert performance.

6.2 Unordered Variants Throughput (Figure 9a,c)

We replicated Figure 2 to evaluate the unordered, vectorized variants at 75% and 85% load factors with 5% updates and 95% lookups. Our results confirm that ZombieHT(V) and AbslHT remain the top performers in this category, achieving significantly higher throughput than CLHT and CuckooHT.

The sole deviation in our reproduction is the complete absence of the `IcebergHT` baseline. As discussed in Section 4.1, `IcebergHT` relies on `AVX512BW` SIMD instructions. Because the `t2.large` CPU lacks this instruction set, the data structure could not be compiled. However, the relative trends of all methods are maintained strongly suggesting the same for IcebergHT as well.

6.3 Home Slot and Tombstone Distances (Figure 10a)

To validate the mechanics behind the throughput results, we successfully reproduced the distribution of home slot distances (Figure 3). The shape of our plotted curves identically matches the paper's findings. ZombieHT(C) maintains a tight distribution where the vast majority of elements are within 100 slots of their ideal hashed location. TombstoneHT exhibits a massive distribution, drifting further and further from its home slot. Meanwhile, RobinHoodHT, ZombieHT, and GraveyardHT maintain tighter bounds. This figure strongly matches the findings of the original authors, showing that we have succesfully reproduced the findings.

6.4 Latency Distributions (Table 5)

Average throughput metrics can obscure the impact of periodic system pauses. Our reproduction of the latency benchmarks, as shown on Table 1, confirmed the consistency claims of Zombie hashing.

In our logs, GraveyardHT demonstrated massive spikes in maximum latency and standard deviation during insert operations, directly corresponding to the periodic $O(n)$ global rebuild sweeps. ZombieHT(C), by performing deamortized, localized redistributions, completely eliminated these spikes, maintaining a bounded maximum latency and a low standard deviation. While the absolute microsecond values in our reproduction were slightly higher due to the slower clock speed of the `t2.large` instance, the multi-order-of-magnitude gap between GraveyardHT's maximum latency and ZombieHT's maximum latency was perfectly preserved. Especially, we can clearly see that GraveyardHT has a 285x higher max latency than ZombieHT(C).

7 Conclusion

In this reproducibility study, we independently verified the core claims presented in "Zombie Hashing: Reanimating Tombstones in a Graveyard." Our experimental results confirm that the authors' novel deamortized redistribution scheme successfully mitigates primary clustering in linear probing hash tables without incurring the periodic, latency-spiking halts characteristic of GraveyardHT. Across heavy churn workloads at 95% load factors, ZombieHT(C) and ZombieHT(V) consistently demonstrated high average throughput and remarkably low maximum latencies.

To successfully execute this replication on a constrained AWS `t2.large` instance, we had to engineer several critical mitigations. By scaling down the hash table quotient allocations, we prevented Out-Of-Memory kernel panics while preserving the data structures' relative behavioral trends.

Ultimately, despite the brittle nature of the benchmarking harness, the underlying Zombie Hashing implementation is robust. Our study proves that the structural advantages of Zombie Hashing hold true even in scaled-down environments, making it a highly practical advancement for real-world data management systems.

Data Availability

All results can be found inside the folder *sponge_paper/*. This folder contains the outputs of various of experiments we ran. Additionally, the python script *generate_figures.py* generates figures 9(a,c), 10(a) and the latency table. These figures can be found in the folder *reproFigures*. The bash script *run_latency.sh* runs all latency experiments (run as is), while the bash script *run_throughput.sh* runs all throughput

Table 1: Reproduced Latency Table 5: Latency distribution of various hash table operations collected over 100 churn cycles. Latency is measured as the time taken to complete 50 operations (μs). 95% LF, 50% Updates. TombstoneHT crashed midway.

Insert (μs)

Percentile	ZombieHT(C)	RobinHoodHT	TombstoneHT	GraveyardHT	ZombieHT(V)	AbslHT	CLHT	CuckooHT
Min	25.52	19.77	—	16.73	8.57	5.89	12.68	12.59
50%	49.86	56.98	—	46.31	23.04	9.36	16.56	29.37
99.99%	581.29	282.87	—	256298.79	97.57	41.53	62.38	138.99
Max	1229.68	499.47	—	349657.39	333.21	208855.16	328.31	623.94
Std dev.	37.21	37.07	—	3098.60	8.03	809.62	2.81	18.13

Deletes (μs)

Percentile	ZombieHT(C)	RobinHoodHT	TombstoneHT	GraveyardHT	ZombieHT(V)	AbslHT	CLHT	CuckooHT
Min	9.83	17.10	—	10.17	6.15	5.98	7.77	9.41
50%	13.09	55.09	—	13.57	9.44	9.60	11.46	11.90
99.99%	52.48	316.80	—	50.95	47.37	43.11	50.14	49.40
Max	213.11	463.10	—	305.86	195.82	289.96	281.24	244.82
Std dev.	2.40	42.28	—	2.25	2.27	2.09	2.24	2.26

Lookup (μs)

Percentile	ZombieHT(C)	RobinHoodHT	TombstoneHT	GraveyardHT	ZombieHT(V)	AbslHT	CLHT	CuckooHT
Min	8.29	7.06	—	8.92	5.48	5.17	5.95	8.76
50%	12.38	9.96	—	12.82	7.43	7.46	8.44	10.86
99.99%	48.64	41.10	—	48.26	39.46	35.76	39.61	48.39
Max	333.63	302.28	—	747.02	210.50	291.98	527.59	348.79
Std dev.	2.48	2.02	—	2.34	1.88	1.72	1.88	2.25

experiments (we selectively run those that we needed). Finally, *reproFigures/figure6a/plot_fig6a.py* outputs the figure 6a plot (in that same folder). Note that in order to run the experiments again, you will have to follow the instructions as per the original repository of the authors [4].

References

[1] AWS Academy. Aws academy platform. https://awsacademy.instructure.com/, 2026.

[2] Michael A. Bender, Bradley C. Kuszmaul, and William Kuszmaul. Linear probing revisited: Tombstones mark the demise of primary clustering. In *2021 IEEE 62nd Annual Symposium on Foundations of Computer Science (FOCS)*, pages 1171–1182, 2022.

[3] Yuvaraj Chesetti, Benwei Shi, Jeff M. Phillips, and Prashant Pandey. Zombie hashing: Reanimating tombstones in a graveyard. *Proc. ACM Manag. Data*, 3(3), 2025.

[4] saltsystemslab. Zombieht codebase. https://github.com/saltsystemslab/ZombieHT, 2026.